

Backdoor Repair Under Shard-Version Skew in Distributed Edge Inference

1st Duong Ngoc Linh Dan

Faculty of Information Technology, Van Lang School of Technology
Van Lang University
Ho Chi Minh City, Vietnam
dan.2374802010091@vanlanguni.vn

2nd Ho Du Tuan Dat

Faculty of Information Technology, Van Lang School of Technology
Van Lang University
Ho Chi Minh City, Vietnam
dat.2374802010097@vanlanguni.vn

3rd Nguyen Minh Chinh

Faculty of Information Technology, Van Lang School of Technology
Van Lang University
Ho Chi Minh City, Vietnam
chinh.2275106050051@vanlanguni.vn

4th Dang Khanh Hung

Faculty of Information Technology, Van Lang School of Technology
Van Lang University
Ho Chi Minh City, Vietnam
hung.dk@vlu.edu.vn (corresponding author)

Abstract—Edge intelligence in 6G networks distributes deep models to edge nodes as independently signed shards that are cached and updated in stages over intermittent links. A per-shard signature proves that a shard is authentic, not that every shard belongs to the same model version. We measure what this gap costs after a backdoor has been repaired, when an adversary who controls the artifact cache re-introduces validly signed pre-repair shards into an otherwise repaired model. Using BadNets on CIFAR-10 with PreActResNet18 partitioned into six shards, we evaluate all 63 rollback subsets against two independently fine-tuned repairs. Rolling back one shard restores at most 16.4% attack success rate (ASR). Recovery depends on which shards are stale rather than on how much of the repair they carry: every hybrid above 50% ASR contains `layer4`, whereas the shard holding 48% of the repair update is dispensable. An adversary who ranks shards by repair share restores between 92.3% and 95.8% ASR with four stale shards while clean accuracy stays within 2.3 points of the repaired model, and the two repairs agree on the 50% classification of 60 of 64 subsets. Epoch-manifest pinning, as specified by TUF and Uptane, rejects every skewed load for 34 bytes per manifest and no measurable verification overhead.

Index Terms—6G edge intelligence, distributed edge inference, backdoor attacks, model supply-chain security, version skew, over-the-air model updates

I. INTRODUCTION

Backdoor repair does not end a model’s lifecycle, for the repaired model must still be distributed, cached, and served. In 6G edge intelligence a model is increasingly partitioned into independently signed shards, so that edge nodes can fetch, cache, and verify each shard separately over intermittent wireless links, and a repair can be rolled out in stages rather than as one atomic replacement [1], [2]. Staged rollout is attractive precisely because it tolerates interruption: a link outage or a handover leaves some nodes on the new version and others on the old, and the deployment continues to serve. Supply-chain protections for this setting verify that each shard is authentically signed. A per-shard signature, however, proves only that a shard is a genuine artifact; it does not prove that the shard belongs to the same repair epoch as the shards around it. An adversary who controls the artifact cache or the update path, without forging a signature or compromising a loader, can supply a validly signed pre-repair shard to a node during an interrupted rollout, and the node then serves a hybrid model that mixes repaired and stale weights.

We measure the machine-learning consequence of this gap rather than propose a protocol. Using BadNets on CIFAR-10 with PreActResNet18 partitioned into six shards, we repair the

infected model by clean fine-tuning under two seeds, evaluate every one of the 63 possible rollback subsets against each repair, and ask when a hybrid restores backdoor behaviour and whether utility monitoring would notice. Three findings follow. Rolling back a single shard is weak: the largest single-shard attack success rate (ASR) is 16.4%, and the clean accuracy (CA) of every single-shard hybrid stays within 6.8 points of the repaired model, so a stale shard is accepted silently and to little effect. Recovery is governed by which shards are stale rather than by how much of the repair they carry: every hybrid above 50% ASR contains `layer4`, whereas `layer3`, which holds 48% of the repair update, is dispensable, and the repair share covered by a subset predicts its ASR only weakly (Spearman $\rho \leq 0.41$). Finally, an adversary who holds both signed versions can rank shards by repair share offline; four shards chosen this way restore 92.3% and 95.8% ASR under the two repairs, at a CA within 2.3 points of the repaired model and within 0.2 points of the infected model that was in service before the repair. The two repairs agree on whether a subset exceeds 50% ASR in 60 of 64 cases.

The defense we evaluate is deliberately not new. Epoch-manifest pinning binds every shard to a per-epoch identifier in signed snapshot metadata, so that a loader rejects a shard as epoch-mismatched instead of accepting it as individually valid; TUF [3] and Uptane [4] specify it for exactly this class of problem. Our contribution is to show that the gap it closes is open in model-shard distribution and to measure what closing it costs: in our serving comparison (Section III-H) it adds 34 bytes of manifest per epoch and no measurable per-load verification cost, which makes it a baseline control rather than a design trade-off.

Section II-A states the threat model. Section II-B describes the dataset, architecture, attack, repairs, and partitioning. Section III reports the single-shard, multi-shard, and shard-selection results for both repairs, the determinism and replication checks, and the serving comparison. Section IV interprets the results and states their limits, Section V relates them to prior work, and Section VI concludes.

II. METHOD AND EXPERIMENTAL SETUP

A. Threat Model

Setting. A model is partitioned into n contiguous shards that are hosted by different edge nodes (split or partitioned inference [1]), a deployment pattern envisioned for 6G edge intelligence [2]. Each shard is signed individually and delivered through an artifact cache, and each node verifies only the shard it hosts. Repairs are rolled out in stages, so an interrupted rollout caused by a link outage or handover is a normal operating condition.

Adversary. The adversary controls only the artifact cache or update path. Its goal is to make the deployment serve a *hybrid* model that mixes repaired shards with validly signed pre-repair shards, restoring the backdoor without failing any per-shard check. It knows the architecture and partition boundaries and holds both signed versions of every shard, which any party that cached the pre-repair artifacts has. It can therefore

compute, offline, how much of the repair each shard carries (Section II-D). It needs no queries to the model and no training data.

Trust assumptions. The coordinator, loader, runtime and any stored minimum-epoch state are trusted; signatures cannot be forged. A fully compromised worker that misreports its loaded weights is out of scope, since it requires attestation.

Per-shard verification proves that each shard is authentic, not that all shards belong to the same epoch. Epoch-manifest pinning, as defined in TUF [3] and Uptane [4], closes this gap without a new protocol. We measure the ML-specific consequence of its absence.

B. Models, Attack, and Repairs

Data, model and attack. We use CIFAR-10 [5] (50,000 training and 10,000 test images) and PreActResNet18 [6]. The infected model is the publicly released BackdoorBench [7] BadNets [8] checkpoint for this pair (poisoning ratio 10%, 3×3 white patch in the bottom-right corner, target class 0); we do not retrain it. Evaluated under its native input normalization it reaches CA = 91.33% and ASR = 95.06%, matching the published values.

Repair (a): global clean fine-tuning. The infected model is fine-tuned for 20 epochs on the full clean training set (SGD, momentum 0.9, weight decay 5×10^{-4} , learning rate 0.01 with cosine annealing, batch 128), updating every parameter. We train two such repairs. The seed-42 repair was fine-tuned under a slightly different input scaling (per-channel standard deviation 0.2023, 0.1994, 0.2010) from the one used for evaluation; its accuracy changes by less than 0.2 points between the two, so this does not affect the results. The seed-0 repair uses the evaluation normalization throughout. They reach CA = 93.18% and ASR = 0.81% (seed 42) and CA = 93.51% and ASR = 1.13% (seed 0). Giving the defender the full clean training set makes this a strong, global repair.

C. Partitioning and Substitution

We partition PreActResNet18 into n contiguous shards along top-level module boundaries. At $n=6$ the shards are the stem (`conv1`), the four residual stages `layer1`–`layer4`, and the classifier head (`linear`), indexed 0–5. At $n=3$ they are {stem, `layer1`}, {`layer2`, `layer3`} and {`layer4`, head}. Rolling back a set of shards replaces all of their tensors, including BatchNorm statistics, with the infected versions and leaves every other shard repaired. Rolling back all n shards reproduces the infected model exactly (ASR = 95.06%, CA = 91.33% at both granularities), which verifies the harness.

D. Repair-Share Metric

The *repair share* of shard s is the fraction of the squared repair update that it carries:

$$\text{share}_s = \frac{\|\delta_s\|_2^2}{\sum_j \|\delta_j\|_2^2}, \quad \delta_s = \theta_s^{\text{rep}} - \theta_s^{\text{inf}}, \quad (1)$$

where θ_s are the learnable parameters of shard s ; BatchNorm buffers (`running_mean`, `running_var`,

`num_batches_tracked`) are excluded because they are not learned. By construction $\sum_s \text{share}_s = 1$, and the share of a set of shards is the sum of their shares. The metric needs only the two signed checkpoints, so an attacker who holds both can compute it offline.

E. Metrics

ASR is the fraction of triggered test images whose true class is not the target that are classified as the target class (9,000 images), following BackdoorBench [7]. **CA** is top-1 accuracy on the 10,000 clean test images. Both are measured in evaluation mode. All models are evaluated with BackdoorBench’s input normalization (per-channel standard deviation 0.247, 0.243, 0.261). With these test-set sizes, the 95% binomial confidence half-width is at most 1.0 percentage point for either metric, so we report differences below 1 point as ties.

III. RESULTS

A. Baseline Reproduction

Table I confirms that our evaluation pipeline reproduces the published BackdoorBench infected checkpoint (BadNets, CIFAR-10), and both fine-tuned repairs suppress the backdoor to below 1.2% ASR while raising clean accuracy above the infected model.

TABLE I
BASELINE MODEL PERFORMANCE ON CIFAR-10 TEST SET.

Model	ASR (%)	CA (%)
Infected (BackdoorBench, BadNets)	95.06	91.33
Repaired, seed 42 (fine-tune)	0.81	93.18
Repaired, seed 0 (fine-tune)	1.13	93.51

B. Sanity Checks: Hybrid Models Are Not Broken

Before evaluating partial rollbacks, we verify that the substitution harness is correct and that the resulting hybrid models remain functional.

Full-rollback reconstruction. Substituting all n shards with their pre-repair counterparts should exactly reconstruct the infected model. Table II confirms this at both granularities: the recovered ASR and CA match the infected baseline exactly, ruling out any bug in the substitution code.

Hybrid models are not broken. Across all partial rollback configurations ($n = 6$, one to five shards substituted), hybrid models achieve clean accuracies between 74.76% and 93.15% (seed 42) and 79.87% to 93.52% (seed 0). No hybrid model degrades to random-chance accuracy ($\approx 10\%$). This is important: it means a deployment that monitors only clean accuracy would observe nothing unusual when a stale shard is silently accepted.

C. Single-Shard Rollback (Negative Result)

We first evaluate whether substituting a single stale shard is sufficient to restore the backdoor. Fig. 1 shows, for each $n=6$ shard, its repair-share $\text{share}_s \times 100$ (blue) alongside the ASR when that shard alone is rolled back, for both repairs (red =

TABLE II
FULL-ROLLBACK VERIFICATION. SUBSTITUTING ALL n SHARDS EXACTLY REPRODUCES THE INFECTED MODEL.

Config	ASR (%)	CA (%)	Single-shard hybrid CA (%)
$n = 3$	95.06	91.33	—
$n = 6$, seed 42	95.06	91.33	86.76–93.15
$n = 6$, seed 0	95.06	91.33	86.78–93.52

seed 42, orange = seed 0). The relative update norm r_s for each shard is given in Table III.

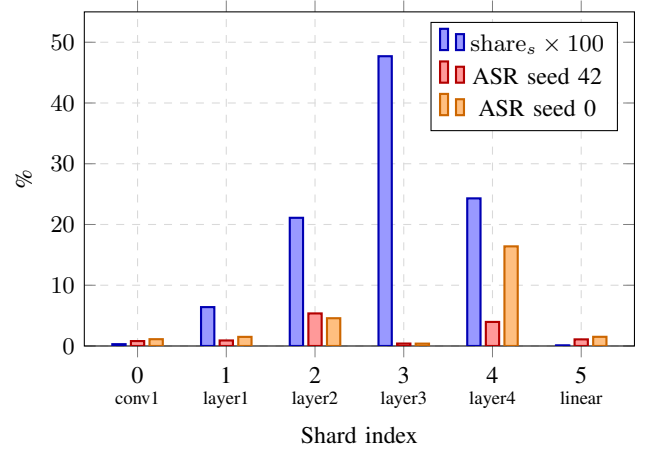


Fig. 1. Per-shard repair share ($\text{share}_s \times 100$, blue) vs. single-shard rollback ASR for seed 42 (red) and seed 0 (orange), $n = 6$. Shard 3 carries the largest share (47.7%) yet yields the *lowest* ASR in both repairs, indicating that repair-share concentration does not predict single-shard rollback efficacy.

TABLE III
RELATIVE UPDATE NORM r_s PER SHARD ($n = 6$, SEED 42).

Shard	Layer(s)	r_s
0	conv1 (stem)	0.158
1	layer1	0.265
2	layer2	0.355
3	layer3	0.395
4	layer4	0.242
5	linear (head)	0.086

We adopt 50% ASR as this study’s criterion for a functionally restored backdoor—the level at which the attack succeeds on a majority of triggered inputs. No individual shard rollback restores the backdoor above the 50% ASR threshold. Across all six shards, the highest single-shard ASR is 5.36% (seed 42, shard 2, layer2) and 16.40% (seed 0, shard 4, layer4). Counterintuitively, shard 3, which carries the largest repair share (47.7%), yields the *lowest* single-shard ASR (0.40% and 0.39% under both repairs), suggesting that the repair is distributed across multiple shards in a way that no single shard can undo.

However, we emphasize the stealth implication: the hybrid model’s clean accuracy (Table II) remains indistinguishable from a normal repaired model at typical monitoring thresholds.

The threat surface exists; it simply requires multiple stale shards, as Section III-D shows.

D. k -Shard Recovery Curve

Fig. 2 shows ASR as k shards are cumulatively rolled back under the repair-aware strategy ($n = 6$), where shards are selected greedily in decreasing order of share_s (i.e., shard 3 first, then shard 4, then shard 2, and so on). The cumulative repair share covered by the first k shards is shown alongside ASR and CA.

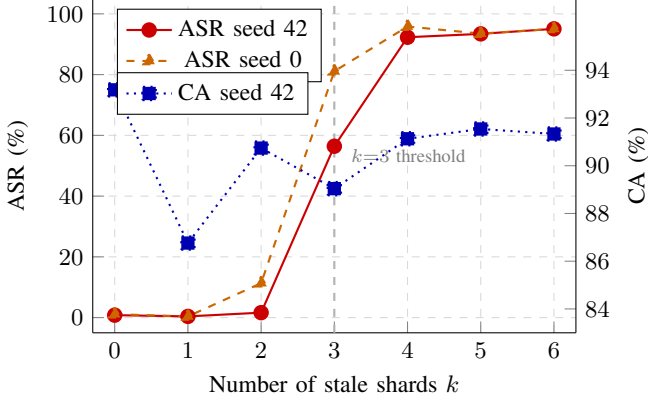


Fig. 2. ASR and CA vs. k stale shards under repair-aware strategy ($n = 6$, greedy by share_s , shards in order 3,4,2,1,0,5). Both repairs cross 50% ASR at $k = 3$. At $k = 4$, CA stays within 2.3 points of the repaired model and within 0.2 points of the infected model.

The curve reveals a sharp threshold at $k = 3$: rolling back three shards (shards 3, 4, and 2)—which together cover 93.1% of the total repair update—causes ASR to jump from 1.63% to 56.42% (seed 42) and from 11.34% to 81.11% (seed 0). Beyond this point ($k \geq 4$), ASR saturates near the fully infected model’s 95.06%. The relationship between cumulative repair share and ASR is thus non-linear: the backdoor is not gradually restored proportionally to coverage, but exhibits a sharp recovery once the repair is sufficiently undermined.

E. Shard-Selection Strategy Comparison

Fig. 3 compares three shard-selection strategies at every k : (a) **repair-aware**: shards selected greedily by decreasing share_s (requires knowledge of the repair update norms); (b) **architectural**: shards selected in index order (0, 1, 2, ...), requiring only knowledge of the partition boundaries; and (c) **exact mean** over all $\binom{6}{k}$ subsets, replacing the earlier five random draws.

Repair-aware selection is the most efficient of the three ordered strategies: it reaches $> 50\%$ ASR at $k = 3$, requiring only three stale shards to restore the backdoor. Architectural selection requires $k = 5$ to cross the same threshold. The exact mean over all $\binom{6}{k}$ subsets crosses 50% ASR at $k = 5$ on average (seed 42) and remains below 50% at $k = 4$ for both repairs. Importantly, architectural selection—which requires no knowledge of repair update norms, only knowledge of the partition indices—still eventually achieves full backdoor recovery at $k = 5$, which shows that the attack does not require

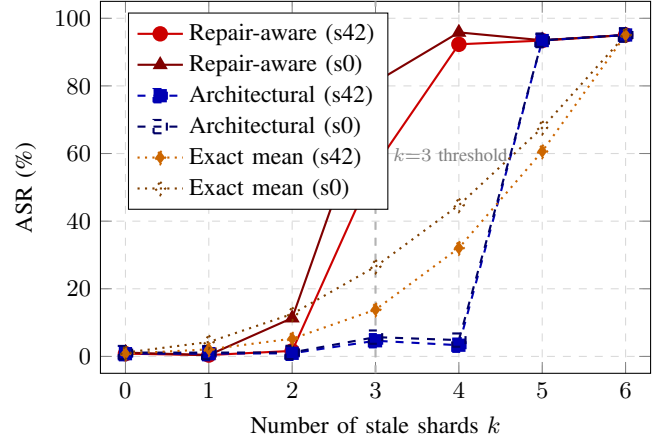


Fig. 3. ASR (%) vs. k stale shards under three selection strategies ($n = 6$, both repairs). Repair-aware is the most efficient of the three ordered strategies, reaching $> 50\%$ ASR at $k = 3$. Architectural requires $k = 5$. The exact mean over all $\binom{6}{k}$ subsets is deterministic and replaces the earlier five random draws.

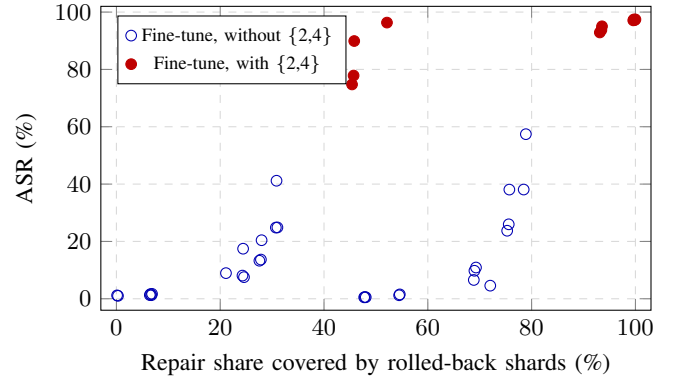


Fig. 4. Repair share covered vs. restored ASR for every rollback subset (all 63 non-empty subsets, fine-tune repair, $n = 6$). Filled markers: subsets containing `layer4`. Share is a weak predictor ($\rho \leq 0.41$); `layer4` membership is the structural discriminator.

privileged access to repair internals to succeed at moderate stale-shard counts.

F. What Drives Recovery: Repair Share and Shard Interaction

Fig. 4 plots, for every shard subset (all 63 non-empty subsets at $n = 6$), the repair share it covers against the ASR it restores. Share alone is a weak predictor (Spearman $\rho = 0.33$ for seed 42, $\rho = 0.41$ for seed 0, computed by `scripts/plot_locality_asr.py`). A single shard never exceeds 5.36% ASR (seed 42) or 16.40% (seed 0), even shard 3, which carries 47.74% of the repair and restores only 0.40%. The key structural finding is that `layer4` is necessary: every hybrid above 50% ASR contains `layer4` in both repairs. `layer3`—the shard with the largest repair share (47.7%)—is dispensable: the worst-case four-shard rollback under seed 0 ($\{1, 2, 4, 5\}$, 96.07% ASR) excludes it entirely. The share-greedy attacker of Section III-E discovers these paths because `layer2`–`layer4` are also the three highest-share shards.

G. Determinism and Sampling Variance

The repair-aware and architectural orders are deterministic, and re-running the sweep under a second evaluation seed (0 instead of 42) reproduced all repeated subset evaluations exactly, so the pipeline has no hidden randomness. This is a determinism check, not a replication over repair seeds. Only the exact mean over all $\binom{6}{k}$ subsets varies between repairs: 13.82% at $k = 3$ (seed 42) and 26.55% (seed 0). The difference arises because seed-0 subsets are more likely to contain `layer4` (Section III-F). To test whether the result depends on the repair itself, we fine-tuned a second repaired model with seed 0 (CA = 93.51%, ASR = 1.13%, trained under BackdoorBench normalization throughout). Its three highest-share shards are the same as seed 42 ($\{3, 4, 2\}$), and rolling them back restores 81.11% ASR at 89.93% CA ($k=3$); at $k=4$ ($\{1, 2, 3, 4\}$), ASR reaches 95.82% at 91.20% CA—within 0.13 points of the infected baseline. The two repairs agree on whether a subset exceeds 50% ASR in 60 of 64 cases (Pearson $r = 0.926$).

H. Deployment and Serving Comparison

All timings below were measured on a single M2 MacBook, on CPU, with each scheme’s loader run as one OS process; this comparison uses the $n = 3$ shard partition, unlike the $n = 6$ partition used elsewhere in the paper. Table IV compares per-shard signature checking—where “signature” denotes HMAC-SHA256 used here as a stand-in for a production signing scheme—against epoch-manifest pinning as defenses against shard-version skew, measured over 30 repetitions per (scheme, scenario) cell against the real infected/repaired checkpoint pair, with each scheme’s loader run as a separate OS process. In the skewed-load scenario – one shard substituted with an older, independently and validly signed version, as a per-shard-signature-only loader would see after an interrupted rollout – per-shard signature checking accepts the substituted shard in all 30 repetitions, since it has no way to detect that the shard belongs to a different repair epoch than its neighbours; epoch-manifest pinning rejects it in all 30 repetitions and correctly identifies the substituted shard. This added protection costs 34 additional bytes of manifest per epoch (848 vs. 814 bytes) to store the epoch identifier, and adds no measurable per-load verification cost once the identifier is hashed via an incremental update rather than a full-buffer concatenation (132.0 ± 3.8 ms vs. 133.4 ± 3.1 ms per load, not statistically distinguishable at this sample size). Recovering from a detected skew – reloading from the correct manifest immediately after rejection – takes 148.4 ± 5.3 ms; per-shard signature checking has no equivalent recovery path, since it never detects the skew to begin with.

IV. DISCUSSION

Which shards are stale matters more than how many. A single stale shard restores at most 16.4% ASR, so the single-shard case is the lower bound of the attack. Recovery is not proportional to the repair share reverted: `layer3` carries 48% of the repair update and its rollback alone yields 0.4% ASR,

TABLE IV
PER-SHARD SIGNATURE CHECKING VS. EPOCH-MANIFEST PINNING
($n = 30$ PER CELL).

Metric	Sig.-only check	Epoch pinning
Manifest size (bytes)	814	848
Verify time / load (ms)	133.4 ± 3.1	132.0 ± 3.8
Rollout recovery time (ms)	N/A (skew undetected)	148.4 ± 5.3
Skewed model accepted?	Yes (undetected)	No (rejected)

whereas every hybrid above 50% ASR contains `layer4`, which carries 24%. The share metric therefore locates the repair but not the backdoor. An adversary who holds both signed epochs can still exploit it, for ranking shards by share and rolling back the top four restores 92.3% and 95.8% ASR under the two repairs, and an adversary who knows only the partition needs five.

Four stale shards is a stealthy attack. The four-shard hybrids above serve at 91.1% and 91.2% CA, which is 2.1 and 2.3 points below the repaired model and within 0.2 points of the infected model that was in service before the repair. A monitor that compares against the pre-repair baseline sees nothing; one that compares against the repaired model needs labelled canary data and an alarm threshold tighter than 2 points. Label-free monitoring would not detect it. Detection therefore belongs at the artifact layer, where provenance is checked, not at the inference layer, where only aggregate accuracy is visible.

Two repairs, one pattern. The two fine-tuned repairs differ in seed and in training normalization, yet their ASR across all 64 subsets correlates at $r = 0.926$ and they agree on the 50% classification of 60 subsets. The four disagreements all contain `layer4` without `layer2`, where the seed-0 repair is the more fragile. Under that repair three hybrids also exceed the infected model’s own ASR, reaching 96.5%, so a partial rollback can be a stronger attack than the original model.

The fix is cheap. Every hybrid shares one root cause: each shard is checked for authenticity, not for membership in the same epoch. Binding every shard to an epoch identifier in signed snapshot metadata, as TUF [3] and Uptane [4] specify, rejected every skewed load in our serving test for 34 extra bytes per manifest. In partitioned edge inference each node sees only its own shard, so the epoch check must be enforced per node against a pinned manifest, not only at the publisher.

A. Limitations

We study one architecture (PreActResNet18), one dataset (CIFAR-10), one attack (BadNets at 10% poisoning), and one family of repair (clean fine-tuning with the full clean training set, a strong defender), with block-aligned partitions only. The two repairs differ in seed and, as stated in Section II-B, in training normalization, so they are two independent repairs rather than a controlled seed study. The serving test uses HMAC-SHA256 as a stand-in for signatures, one skew scenario at $n = 3$, and single-machine timings. Local repairs,

other triggers and architectures, and non-aligned partitions are future work.

V. RELATED WORK

Backdoor repair and its fragility. Fine-tuning and pruning defenses such as fine-pruning [9] and ANP [10] remove backdoors after training. Min et al. [11] show that models purified by several such defenses, ANP among them, quickly re-learn the backdoor when retuned on a few poisoned samples or probed with crafted queries. DeferBad [12] designs the backdoor so that unlearning leaves a residue that later benign fine-tuning reactivates. Both need training or queries. Our attacker needs neither: it restores backdoor behaviour purely by serving shards that were once legitimately signed.

Weight substitution and merging. The subnet replacement attack [13] overwrites part of a deployed model with weights trained for the purpose, and BadMerging [14] plants backdoors through merging of distinct models. Our attacker cannot craft weights; it reuses the two endpoints of one repair.

Artifact integrity and update security. Sentry [15] authenticates ML artifacts as they are loaded, and the OpenSSF Model Signing specification [16] signs one manifest of per-file digests. Both prove that each artifact is authentic. When shards are verified independently by the nodes that host them, as in partitioned edge inference [1], [2], authenticity alone does not stop a node from accepting a stale but genuine shard. TUF [3] and Uptane [4] define the rollback and mix-and-match protections that close this gap; we quantify what their absence costs for a repaired model.

VI. CONCLUSION

Signed but stale shards can undo a backdoor repair in partitioned edge inference. Against a strong fine-tuning repair, one stale shard restores at most 16.4% ASR, but four shards chosen by repair share restore 92.3% to 95.8% ASR at a clean accuracy within 2.3 points of the repaired model, and recovery hinges on `layer4` being stale rather than on the fraction of the repair reverted. Two independent repairs agree on this pattern in 60 of 64 rollback subsets. Epoch-manifest pinning rejects these hybrids for 34 bytes per manifest and should be a baseline control wherever 6G edge nodes receive model shards through caches.

REFERENCES

- [1] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,” in *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017, pp. 615–629.
- [2] K. B. Letaief, Y. Shi, J. Lu, and J. Lu, “Edge artificial intelligence for 6G: Vision, enabling technologies, and applications,” *IEEE Journal on Selected Areas in Communications*, vol. 40, no. 1, pp. 5–36, 2022.
- [3] J. Samuel, N. Mathewson, J. Cappos, and R. Dingledine, “Survivable key compromise in software update systems,” in *ACM Conference on Computer and Communications Security (CCS)*, 2010, pp. 61–72.
- [4] T. K. Kuppusamy, L. A. DeLong, and J. Cappos, “Uptane: Security and customizability of software updates for vehicles,” *IEEE Vehicular Technology Magazine*, vol. 13, no. 1, pp. 66–73, 2018.
- [5] A. Krizhevsky and G. Hinton, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009.

- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Identity mappings in deep residual networks,” in *European Conference on Computer Vision (ECCV)*, 2016, pp. 630–645.
- [7] B. Wu, H. Chen, M. Zhang, Z. Zhu, S. Wei, D. Yuan, and C. Shen, “BackdoorBench: A comprehensive benchmark of backdoor learning,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 35, 2022, pp. 10 546–10 559.
- [8] T. Gu, K. Liu, B. Dolan-Gavitt, and S. Garg, “BadNets: Evaluating backdooring attacks on deep neural networks,” *IEEE Access*, vol. 7, pp. 47 230–47 244, 2019.
- [9] K. Liu, B. Dolan-Gavitt, and S. Garg, “Fine-pruning: Defending against backdooring attacks on deep neural networks,” in *Research in Attacks, Intrusions, and Defenses (RAID)*, ser. LNCS, vol. 11050, 2018, pp. 273–294.
- [10] D. Wu and Y. Wang, “Adversarial neuron pruning purifies backdoored deep models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.
- [11] R. Min, Z. Qin, N. L. Zhang, L. Shen, and M. Cheng, “Uncovering, explaining, and mitigating the superficial safety of backdoor defense,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [12] J. Shin and S. Park, “Unlearn to relearn backdoors: Deferred backdoor functionality attacks on deep learning models,” arXiv:2411.14449, 2024.
- [13] X. Qi, T. Xie, R. Pan, J. Zhu, Y. Yang, and K. Bu, “Towards practical deployment-stage backdoor attack on deep neural networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [14] J. Zhang, J. Chi, Z. Li, K. Cai, Y. Zhang, and Y. Tian, “BadMerging: Backdoor attacks against model merging,” in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024.
- [15] A. Gan and Z. Ghodsi, “Sentry: Authenticating machine learning artifacts on the fly,” in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025.
- [16] OpenSSF AI/ML Security Working Group, “OpenSSF model signing (OMS) specification,” <https://github.com/ossf/model-signing-spec>, 2025.